4 网络优化与正则化

难点问题

优化	泛化
经验风险最小	降低模型复杂度

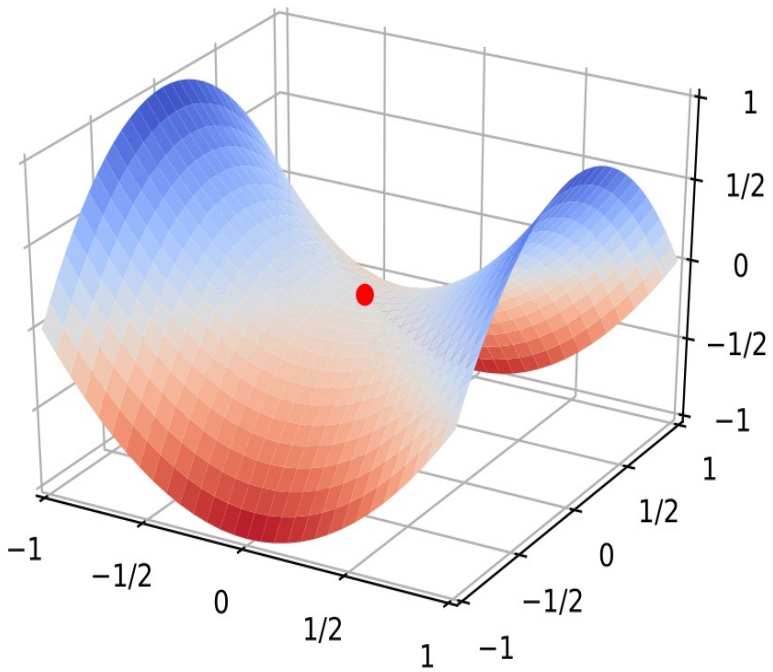
4.1 网络优化

网络优化

- 网络优化是指寻找一个神经网络模型来使得经验风险最小化的过程，包括模型选择以及参数学习等。
- 难点：
  - 结构差异大
    - 没有通用的优化算法
    - 超参数多
  - 非凸优化问题
  - 梯度消失（爆炸）问题

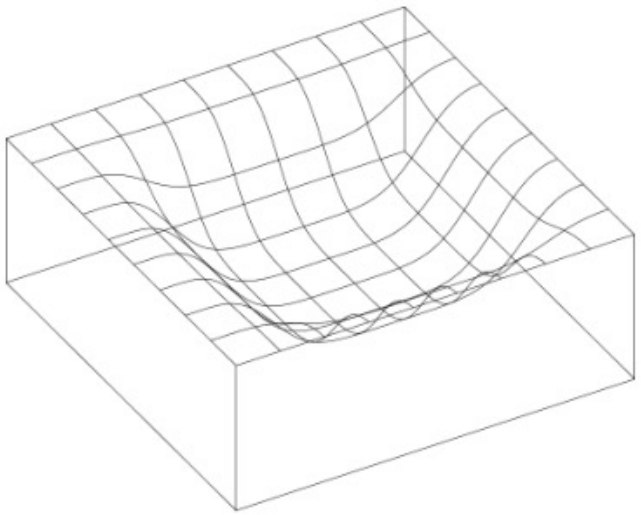
高维变量的非凸优化问题

- 在高维空间中，非凸优化的难点在于如何逃离鞍点（Saddle Point）。
- 鞍点的梯度是0，但是在一些维度上是最高点，在另一些维度上是最低点。

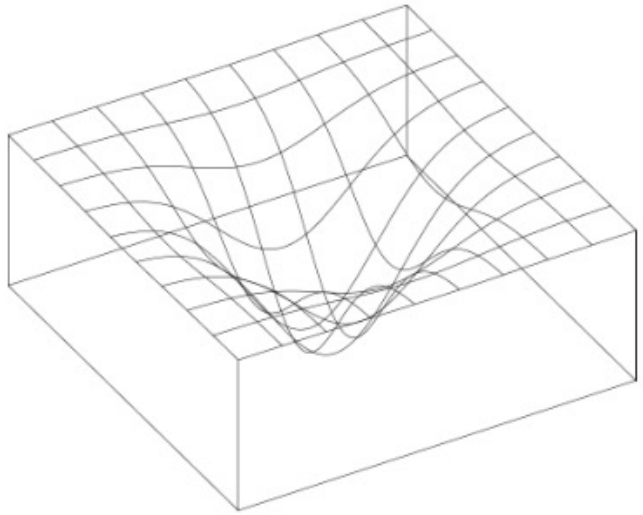


平坦最小值

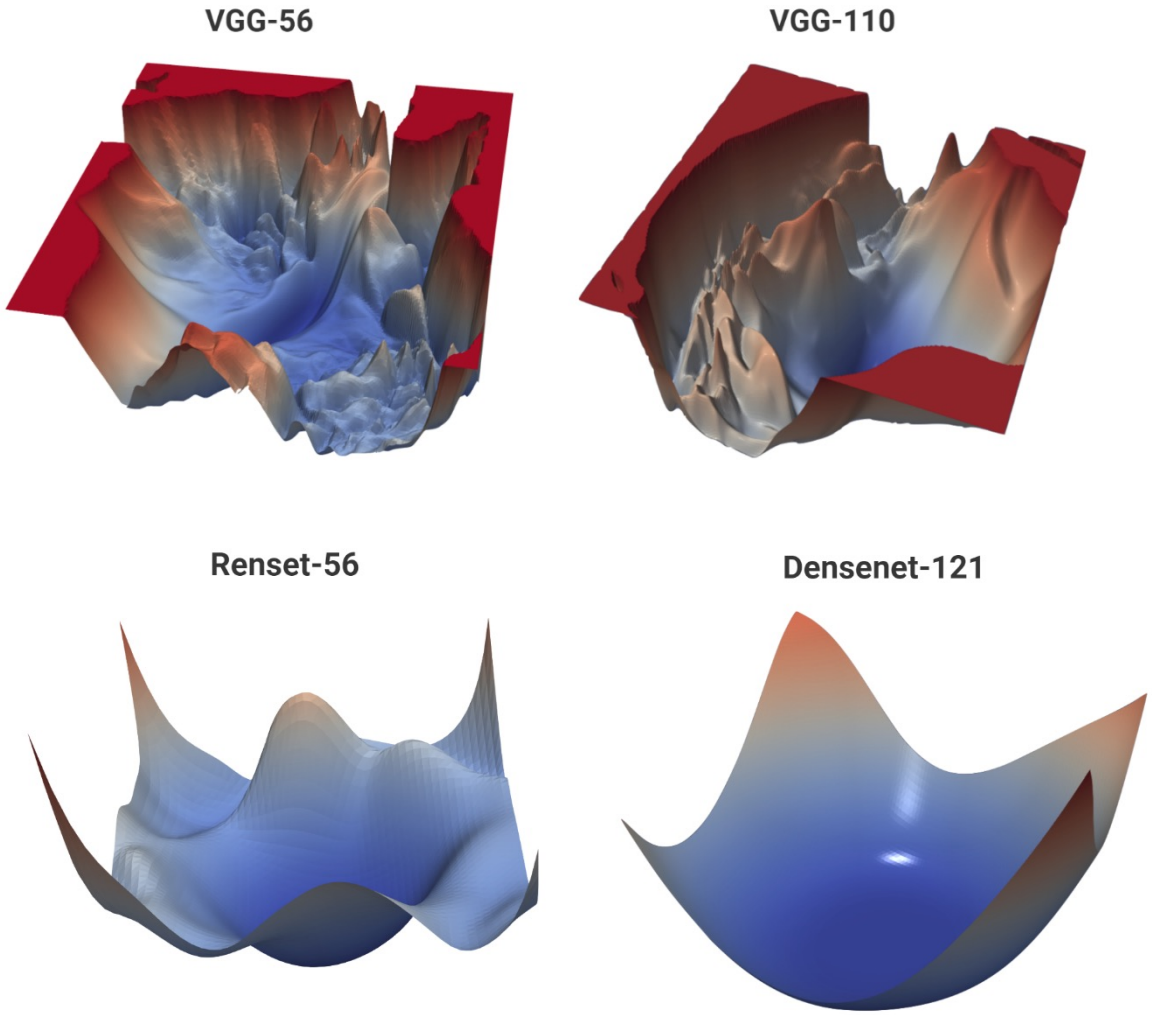
- 在一个平坦最小值的邻域内，所有点对应的训练损失都比较接近。
- 大部分的局部最小解是等价的。
- 局部最小解对应的训练损失都可能非常接近于全局最小解对应的训练损失。



(a) 平坦最小值



(b) 尖锐最小值



没有跳接

有跳接

Li H, Xu Z, Taylor G, et al. Visualizing the loss landscape of neural nets[C]//Advances in Neural Information Processing Systems. 2018: 6389-6399.

神经网络优化的改善方法

- 更有效的优化算法来提高优化方法的效率和稳定性
  - 动态学习率调整
  - 梯度估计修正
- 更好的参数初始化方法、数据预处理方法来提高优化效率
- 修改网络结构来得到更好的优化地形
  - 优化地形指在高维空间中损失函数的曲面形状，好的优化地形通常比较平滑
  - 使用 ReLU 激活函数、残差连接、逐层归一化等
- 使用更好的超参数优化方法

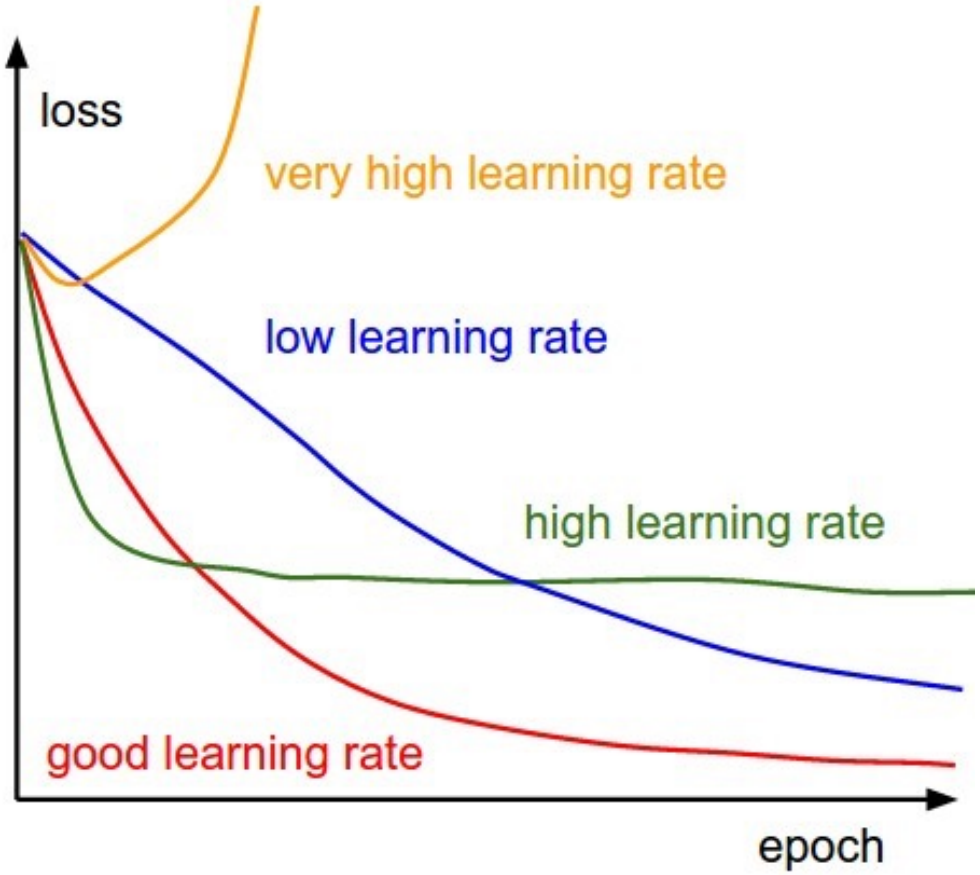
优化算法改进

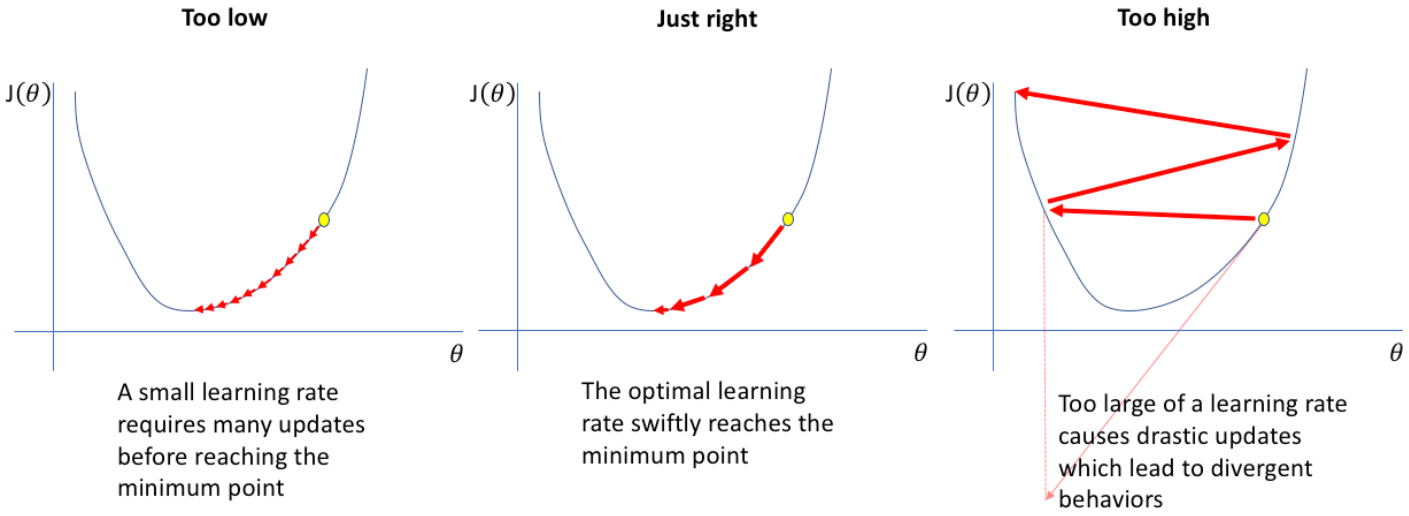
- 深度神经网络的参数学习主要是通过梯度下降法来寻找一组可以最小化结构风险的参数。
- 在具体实现中，梯度下降法可以分为：批量梯度下降、随机梯度下降以及小批量梯度下降三种形式。
- 关键因素：
  - 小批量样本数量
  - 梯度
  - 学习率

改进

- 小批量梯度下降
- 学习率调整
  - 学习率衰减
  - Adagrad
  - Adadelta
  - RMSprop
- 梯度
  - 动量法
  - Nesterov加速梯度
  - 梯度截断

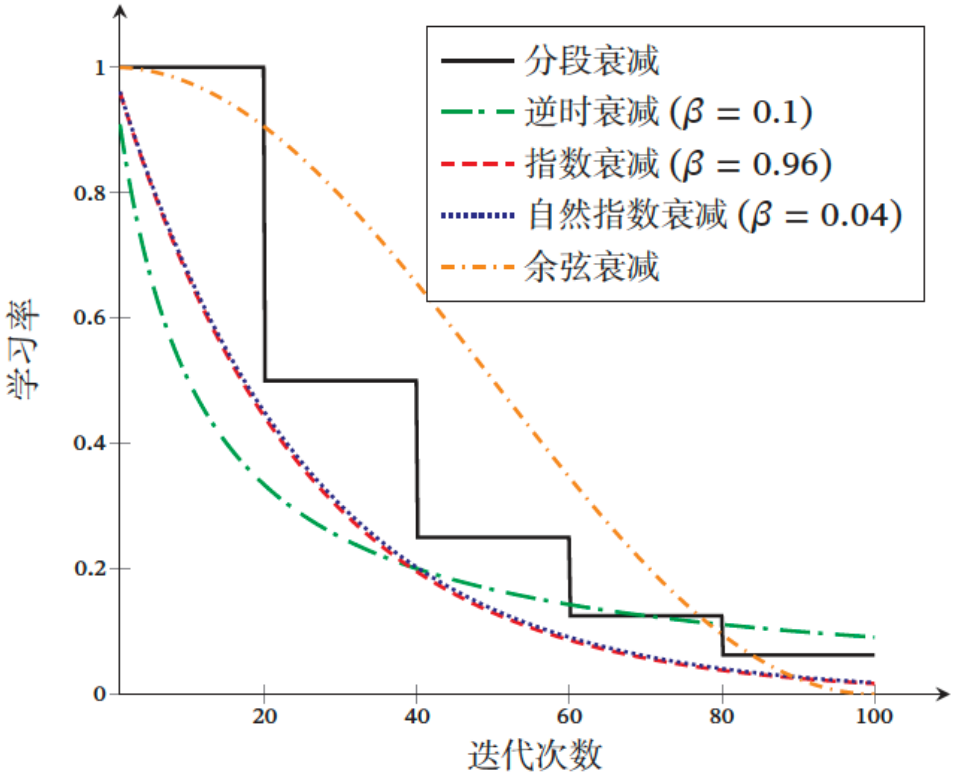
学习率的影响


loss
very high learning rate
low learning rate
high learning rate
good learning rate
epoch


Too low
Just right
Too high
J(θ)
θ
A small learning rate requires many updates before reaching the minimum point
The optimal learning rate swiftly reaches the minimum point
Too large of a learning rate causes drastic updates which lead to divergent behaviors

学习率衰减

- 从经验上看，学习率在一开始要保持大些来保证收敛速度，在收敛到最优点附近时要小些以避免来回振荡。
- 假设设初始化学习率为α_0，在第t次迭代时的学习率α_t。常见的衰减方法有以下几种：
  - 分段常数衰减
  - 逆时衰减 $\alpha_t = \alpha_0 \frac{1}{1+\beta t}$
  - 指数衰减 $\alpha_t = \alpha_0 \beta^t$
  - 自然指数衰减 $\alpha_t = \alpha_0 exp(-\beta t)$
  - 余弦衰减 $\alpha_t = \frac{1}{2}\alpha_0 \left(1 + cos\left(\frac{t\pi}{T}\right)\right)$



学习率预热

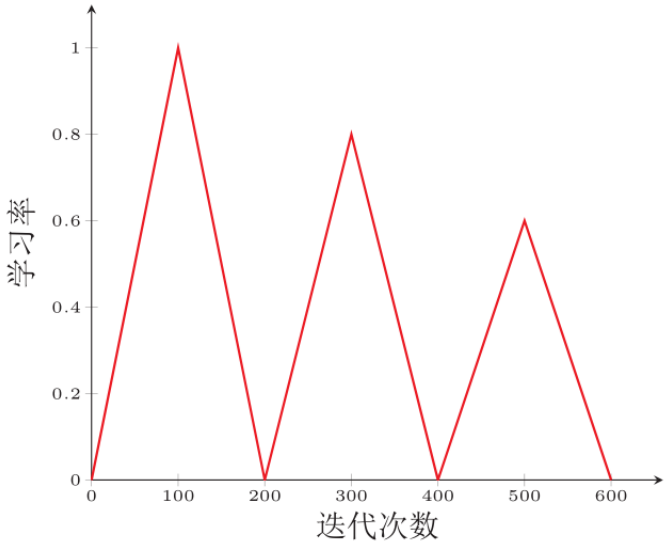
- 刚开始训练时，由于参数是随机初始化的，梯度往往比较大，较大的初始学习率会使训练不稳定。
- 学习率预热：在最初几轮迭代时，采用比较小的学习率，等梯度下降到一定程度后再恢复到初始的学习率。
- 假设预热的迭代次数为T'，初始学习率为α_0，在预热过程中，每次更新的学习率为

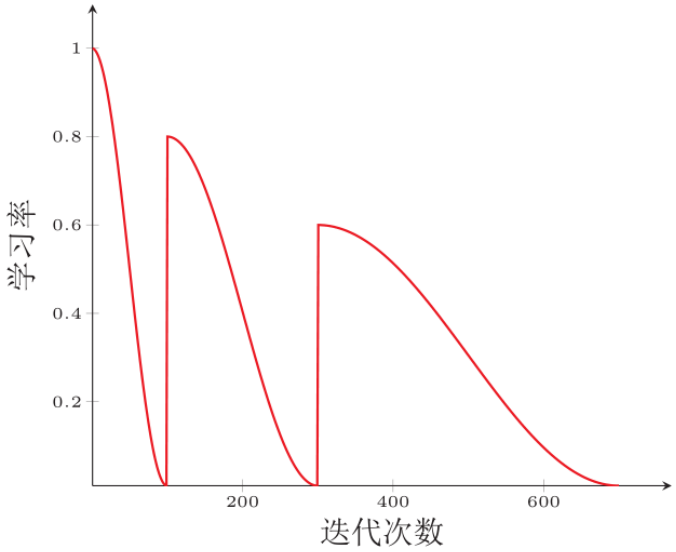
$$\alpha_t' = \frac{t}{T'}\alpha_0$$

当预热过程结束，再选择一种学习率衰减方法来逐渐降低学习率。

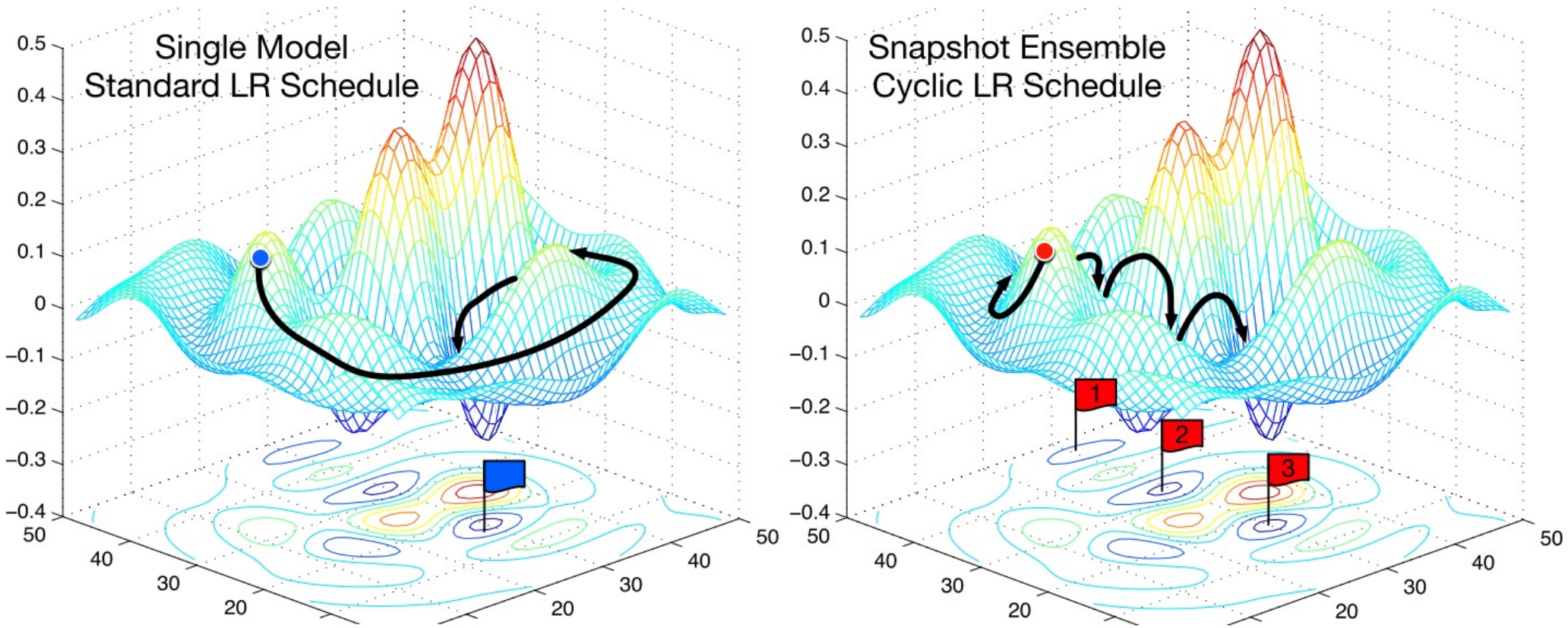
周期性学习率调整

- 为了使得梯度下降法能够逃离鞍点或尖锐最小值，可在训练过程中周期性地增大学习率。
- 两种常用的周期性调整学习率的方法：
  - 循环学习率
  - 带热重启的随机梯度下降





周期性学习率调整


Single Model
Standard LR Schedule
Snapshot Ensemble
Cyclic LR Schedule
1
2
3

自适应学习率

Adagrad

RMSprop

AdaDelta

$$\Delta\theta_t = -\frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \boldsymbol{g}_t$$

$$G_t = \sum_{\tau=1}^{t} \boldsymbol{g}_\tau \odot \boldsymbol{g}_\tau$$

$$G_t = \beta G_{t-1} + (1-\beta)\boldsymbol{g}_t \odot \boldsymbol{g}_t$$

$$= (1-\beta)\sum_{\tau=1}^{t} \beta^{t-\tau} \boldsymbol{g}_\tau \odot \boldsymbol{g}_\tau$$

$$\Delta\theta_t = -\frac{\sqrt{\Delta X_{t-1}^2 + \epsilon}}{\sqrt{G_t + \epsilon}} \mathbf{g}_t$$

$$\Delta X_{t-1}^2 = \beta_1 \Delta X_{t-2}^2 + (1-\beta_1)\Delta\theta_{t-1} \odot \Delta\theta_{t-1}$$

梯度估计修正

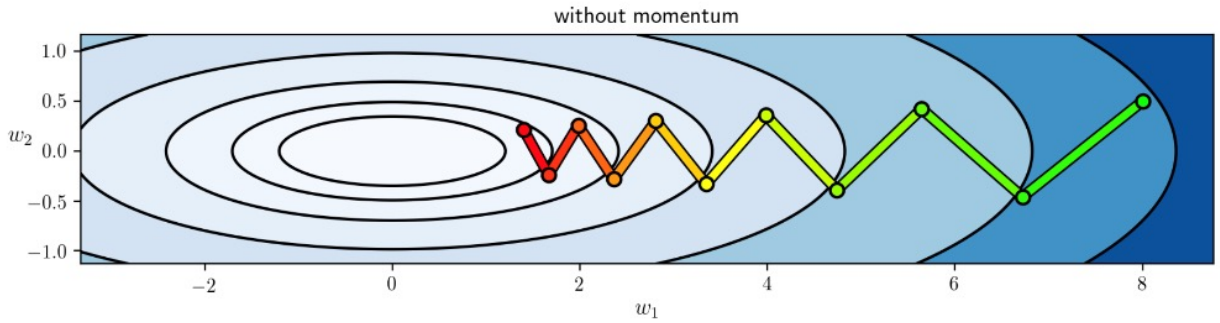
- 在随机（小批量）梯度下降法中，如果每次选取样本数量比较小，每次迭代的梯度估计和整个训练集上的最优梯度并不一致，具有一定的随机性。
- 一种有效地缓解梯度估计随机性的方式是通过使用最近一段时间内的平均梯度来代替当前时刻的随机梯度来作为参数更新的方向，从而提高优化速度。

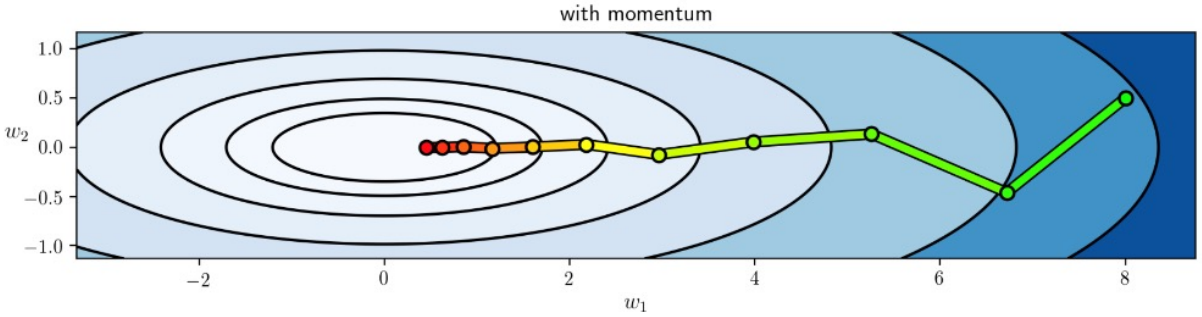
动量法

- 动量法（Momentum Method）：用之前积累动量来替代真正的梯度。每次迭代的梯度可以看作是加速度。
- 在第t次迭代时，计算负梯度的“加权移动平均”作为参数的更新方向，

$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\boldsymbol{g}_t = -\alpha\sum_{\tau=1}^{t}\rho^{t-\tau}\boldsymbol{g}_\tau$$

其中ρ为动量因子，通常设为0.9，α为学习率。





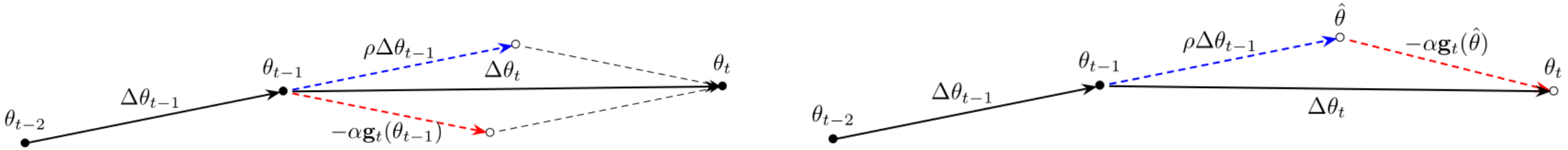
Nesterov加速梯度

- 动量法：先根据 $\Delta\theta_{t-1}$ 更新一次得到参数 $\hat{\theta}$ 再用 $-\boldsymbol{g}_t$ 进行更新。

$$\hat{\theta} = \theta_{t-1} + \rho\Delta\theta_{t-1}, \qquad \theta_t = \hat{\theta} - \alpha\boldsymbol{g}_t$$

其中梯度 $\boldsymbol{g}_t$ 为点 θ_{t-1} 上的梯度。更合理的更新方向应该为 $\hat{\theta}$ 上的梯度。

- Nesterov加速梯度：$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\boldsymbol{g}_t(\theta_{t-1} + \rho\Delta\theta_{t-1})$，其中$\boldsymbol{g}_t(\theta_{t-1} + \rho\Delta\theta_{t-1})$表示损失函数在点 $\hat{\theta} = \theta_{t-1} + \rho\Delta\theta_{t-1}$ 上的偏导数。



(a) 动量法　　(b) Nesterov 加速梯度

Adam算法

- Adam算法（Adaptive Moment Estimation Algorithm）可以看作动量法和RMSprop 算法的结合，不但使用动量作为参数更新方向，而且可以自适应调整学习率。
- 先计算两个移动平均

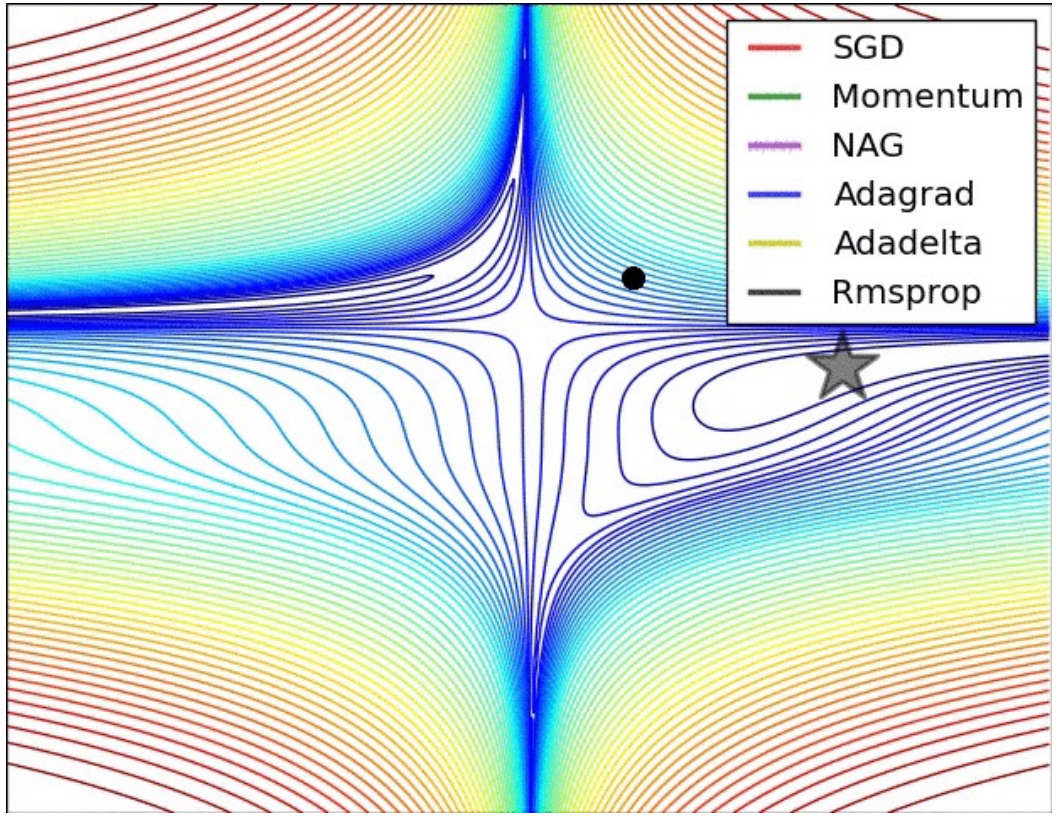
$$M_t = \beta_1 M_{t-1} + (1-\beta_1)\mathbf{g}_t,$$
$$G_t = \beta_2 G_{t-1} + (1-\beta_2)\mathbf{g}_t \odot \mathbf{g}_t,$$

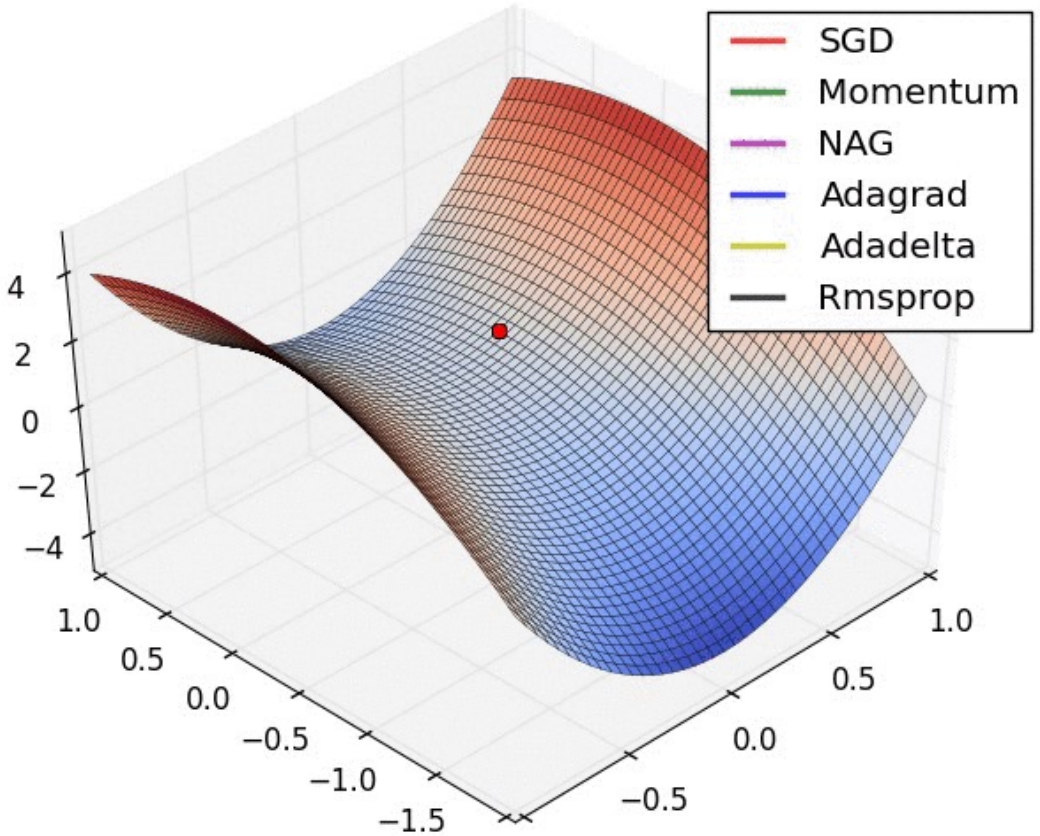
- 偏差修正

$$\hat{M}_t = \frac{M_t}{1-\beta_1^t}, \qquad \hat{G}_t = \frac{G_t}{1-\beta_2^t}.$$

- 更新

$$\Delta\theta_t = -\frac{\alpha}{\sqrt{\hat{G}_t + \epsilon}}\hat{M}_t$$


SGD
Momentum
NAG
Adagrad
Adadelta
Rmsprop


SGD
Momentum
NAG
Adagrad
Adadelta
Rmsprop
4
2
0
−2
−4
1.0
0.5
0.0
−0.5
−1.0
−1.5
1.0
0.5
0.0
−0.5

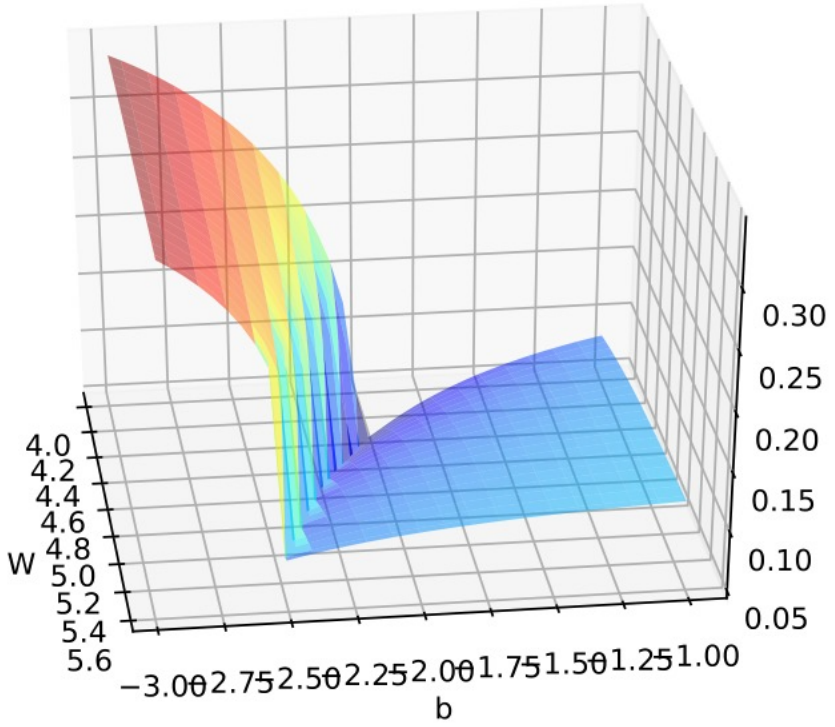
梯度截断

- 梯度截断：把梯度的模限定在一个区间，当梯度的模小于或大于这个区间时就进行截断。
- 按值截断

$$\mathbf{g}_t = \max(\min(\mathbf{g}_t, b), a)$$

- 按模截断

$$\mathbf{g}_t = \frac{b}{\|\mathbf{g}_t\|}\mathbf{g}_t$$



优化算法改进小结

- 大部分优化算法可以使用下面公式来统一描述概括：

$$\Delta\theta_t = -\frac{\alpha_t}{\sqrt{G_t + \epsilon}} M_t,$$

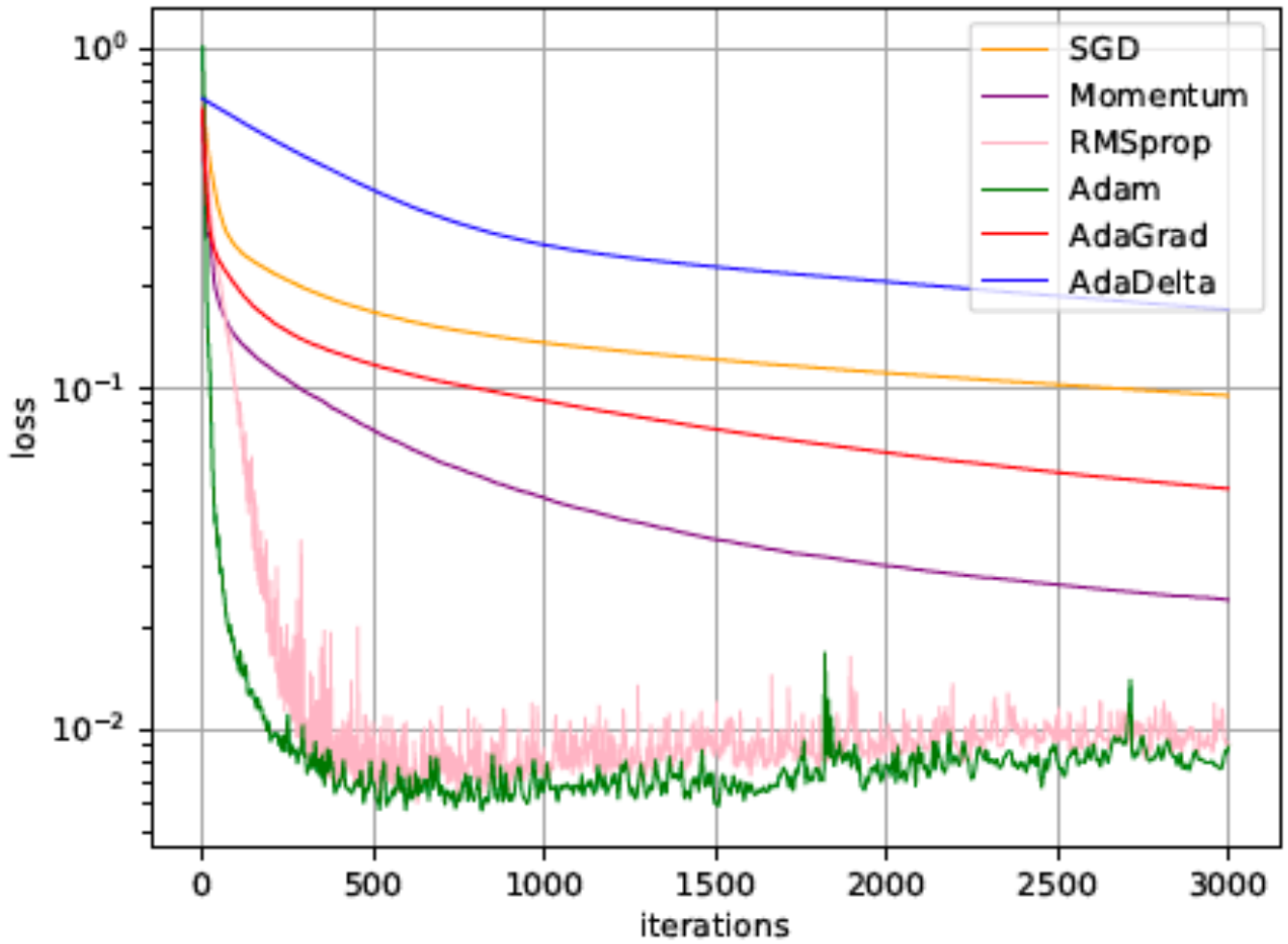
$$G_t = \psi(\mathbf{g}_1, \cdots, \mathbf{g}_t),$$

$$M_t = \phi(\mathbf{g}_1, \cdots, \mathbf{g}_t),$$

g_t 为第t步的梯度

α_t 为第t步的学习率

类别		优化算法
学习率调整	固定衰减学习率	分段常数衰减、逆时衰减、(自然)指数衰减、余弦衰减
	周期性学习率	循环学习率、SGDR
	自适应学习率	AdaGrad、RMSprop、AdaDelta
梯度估计修正		动量法、Nesterov 加速梯度、梯度截断
综合方法		Adam≈动量法+RMSprop



不同优化方法在MNIST 数据集上收敛性的比较

参数初始化

- 神经网络的参数学习是一个非凸优化问题。当使用梯度下降法来进行优化网络参数时，参数初始值的选取十分关键，关系到网络的优化效率和泛化能力。
- 参数初始化方法：
  - 固定值初始化
  - 随机初始化
  - 预训练初始化（pre-trained initialization）

随机初始化

- 基于固定方差的随机初始化方法
  - 高斯分布初始化
    - 使用一个高斯分布$N(0, \sigma^2)$对每个参数进行随机初始化
  - 均匀分布初始化
    - 在一个给定的区间$[-r, r]$ 内采用均匀分布来初始化
- 基于方差缩放的参数初始化
- 正交初始化

基于方差缩放的参数初始化

初始化方法	激活函数	均匀分布 $[-r, r]$	高斯分布 $\mathcal{N}(0, \sigma^2)$
Xavier 初始化	Logistic	$r = 4\sqrt{\frac{6}{M_{l-1}+M_l}}$	$\sigma^2 = 16 \times \frac{2}{M_{l-1}+M_l}$
Xavier 初始化	Tanh	$r = \sqrt{\frac{6}{M_{l-1}+M_l}}$	$\sigma^2 = \frac{2}{M_{l-1}+M_l}$
He 初始化	ReLU	$r = \sqrt{\frac{6}{M_{l-1}}}$	$\sigma^2 = \frac{2}{M_{l-1}}$

正交初始化

- 范数保持性
  - 一个L层的等宽线性网络

$$\boldsymbol{y} = \boldsymbol{W}^{(L)}\boldsymbol{W}^{(L-1)}\dots\boldsymbol{W}^{(1)}\boldsymbol{x}$$

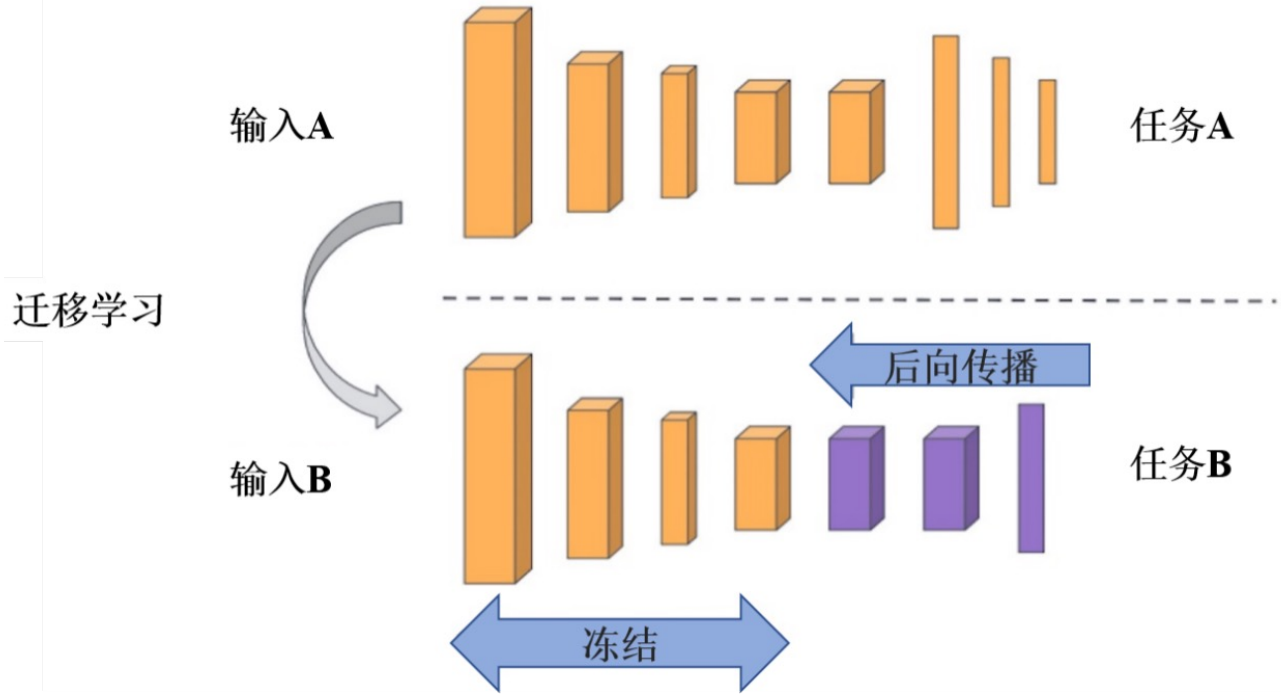
  - 为了避免梯度消失或梯度爆炸问题，我们希望误差项在反向传播中具有范数保持性

$$||\delta^{(l-1)}||^2 = ||\delta^{(l)}||^2 = ||(\boldsymbol{W}^{(l)})^{\top}\delta^{(l)}||^2$$

- 正交初始化：将$\boldsymbol{W}^{(l)}$初始化为正交矩阵，即$\boldsymbol{W}^{(l)}{\boldsymbol{W}^{(l)}}^T = I$

预训练初始化

- 迁移学习（Transfer Learning）是指将某个领域或任务上学习到的知识或模式应用到不同，但相关的领域或问题中。
- 站在前人的肩膀上，用别人的模型、参数。



数据预处理

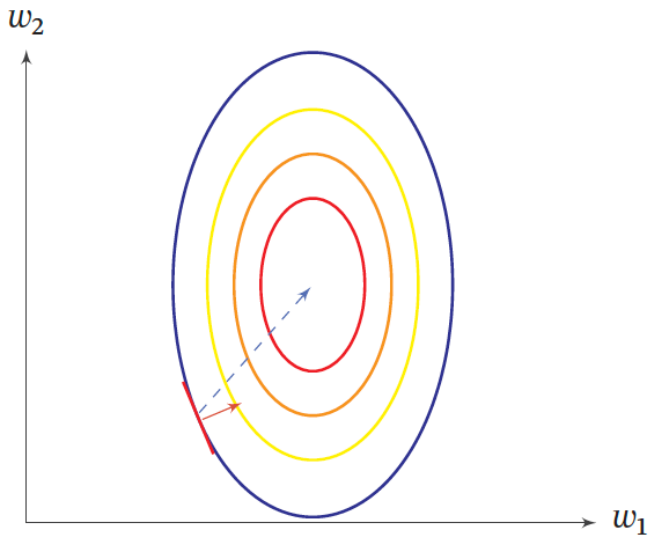
- 数据归一化方法：
  - 最小最大值归一化

$$\hat{x}^{(n)} = \frac{x^{(n)} - \min_n(x^{(n)})}{\max_n(x^{(n)}) - \min_n(x^{(n)})},$$

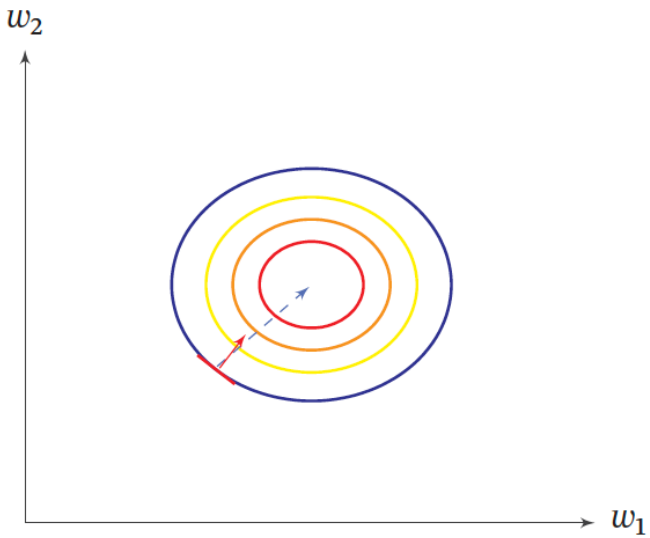
  - 标准化

$$\hat{x}^{(n)} = \frac{x^{(n)} - \mu}{\sigma},$$

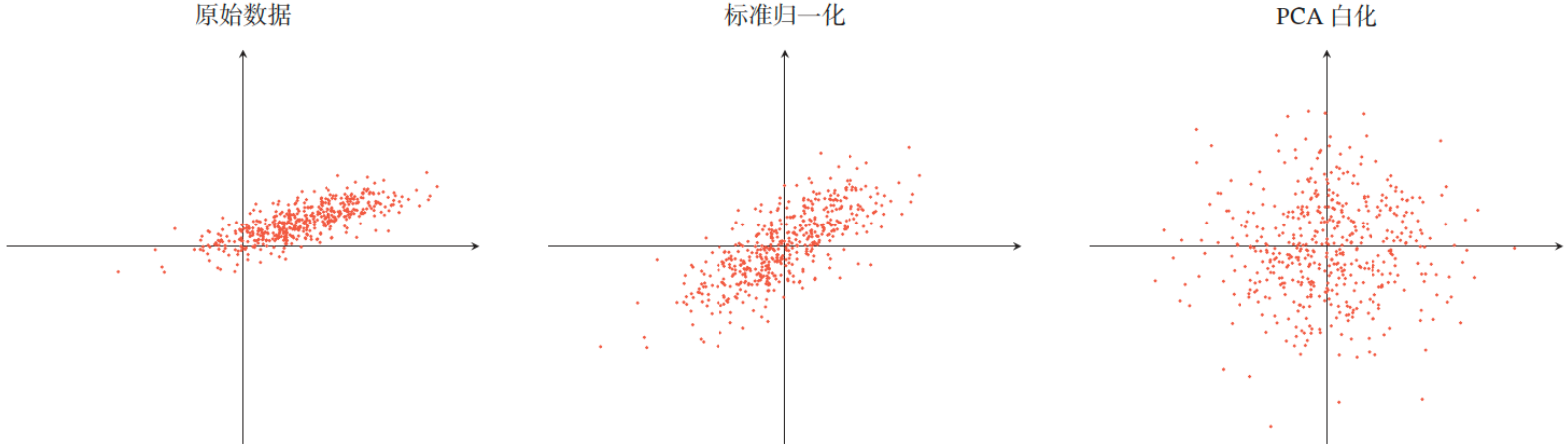
  - 白化：PCA



(a) 未归一化数据的梯度



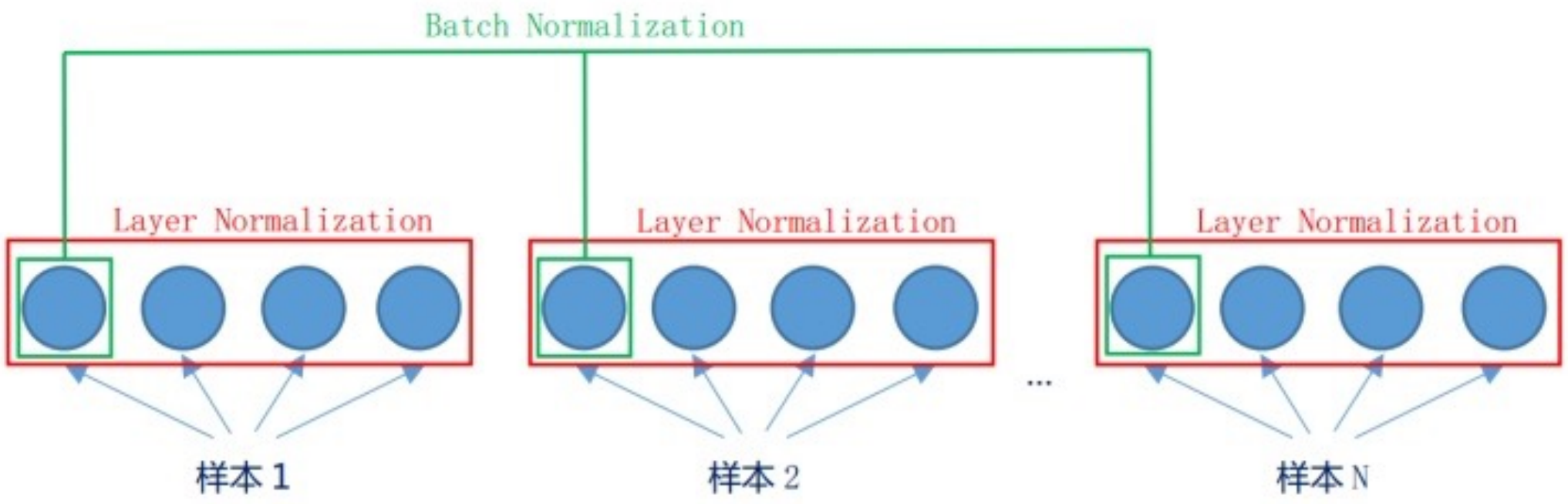
(b) 归一化数据的梯度



逐层归一化

- 目的
  - 更好的尺度不变性
  - 更平滑的优化地形
- 归一化方法
  - 批量归一化（Batch Normalization，BN）
  - 层归一化（Layer Normalization）
  - 权重归一化（Weight Normalization）
  - 局部响应归一化（Local Response Normalization，LRN）

批量归一化VS层归一化



超参数优化

- 超参数：
  - 网络结构：神经元之间的连接关系、层数、每层的神经元数量、激活函数的类型等
  - 优化参数：优化方法、学习率、小批量的样本数量等
  - 正则化系数
- 优化方法：
  - 网格搜索、随机搜索、贝叶斯优化、动态资源分配和神经架构搜索

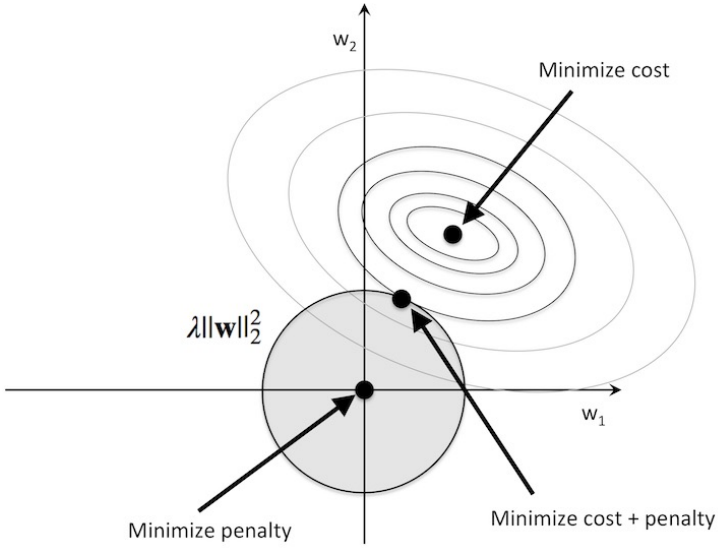
4.2 网络正则化

正则化

- 正则化（Regularization）是一类通过限制模型复杂度，从而避免过拟合，提高泛化能力的方法。

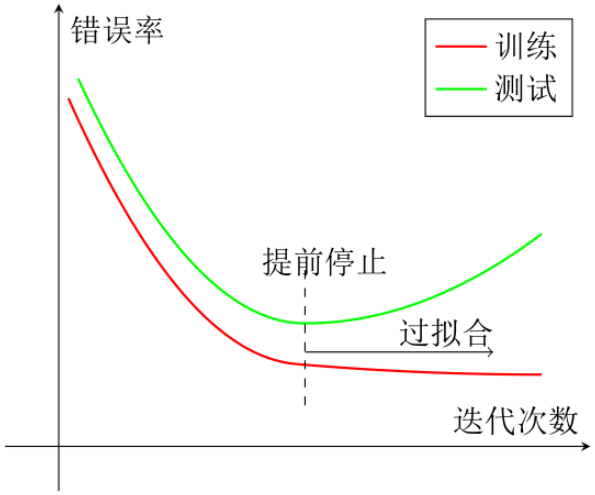
增加优化约束

L1/L2约束、数据增强



干扰优化过程

权重衰减、随机梯度下降、提前停止

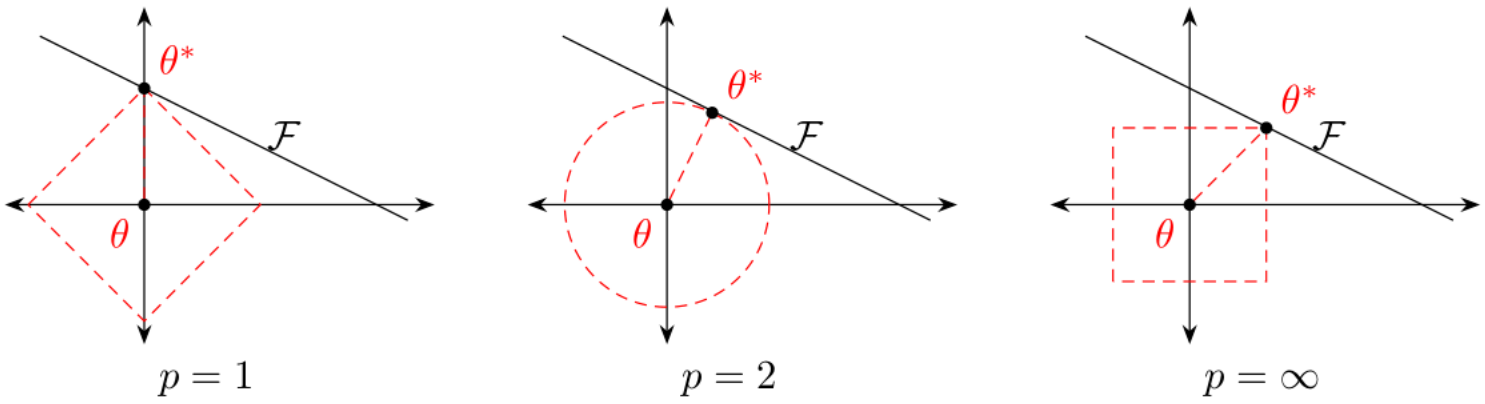


ℓ_1和ℓ_2正则化

- 优化问题可以写为

$$\theta^* = \arg\min_{\theta} \frac{1}{N} \sum_{n=1}^{N} \mathcal{L}\big(y^{(n)}, f(\mathbf{x}^{(n)}, \theta)\big) + \lambda \ell_p(\theta)$$

  - ℓ_{p} 为范数函数，p的取值通常为{1,2}代表ℓ_1和ℓ_2范数，λ为正则化系数。



权重衰减

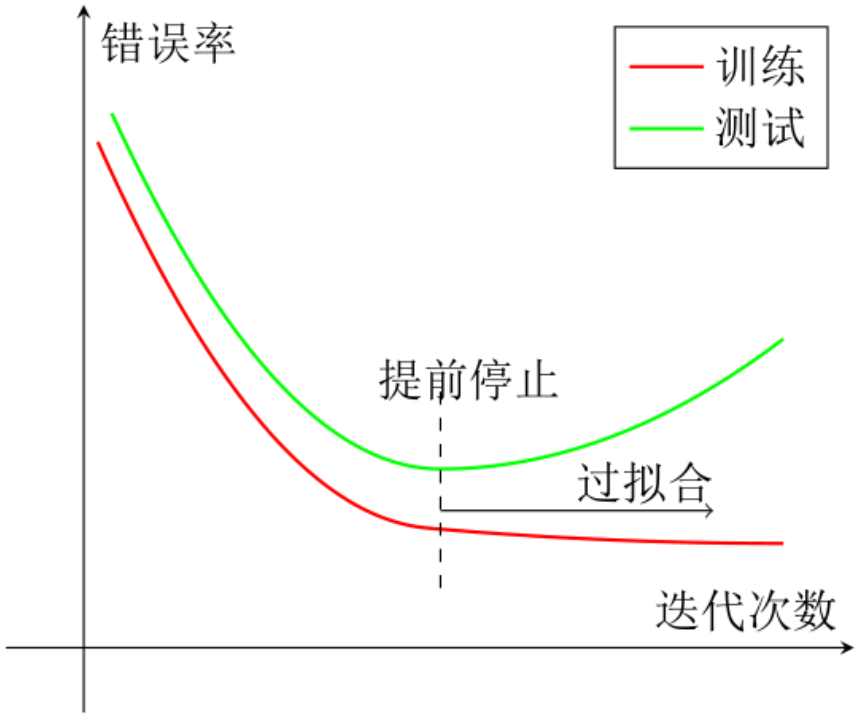
- 在每次参数更新时，引入一个衰减系数w。

$$\theta_t \leftarrow (1-w)\theta_{t-1} - \alpha \mathbf{g}_t$$

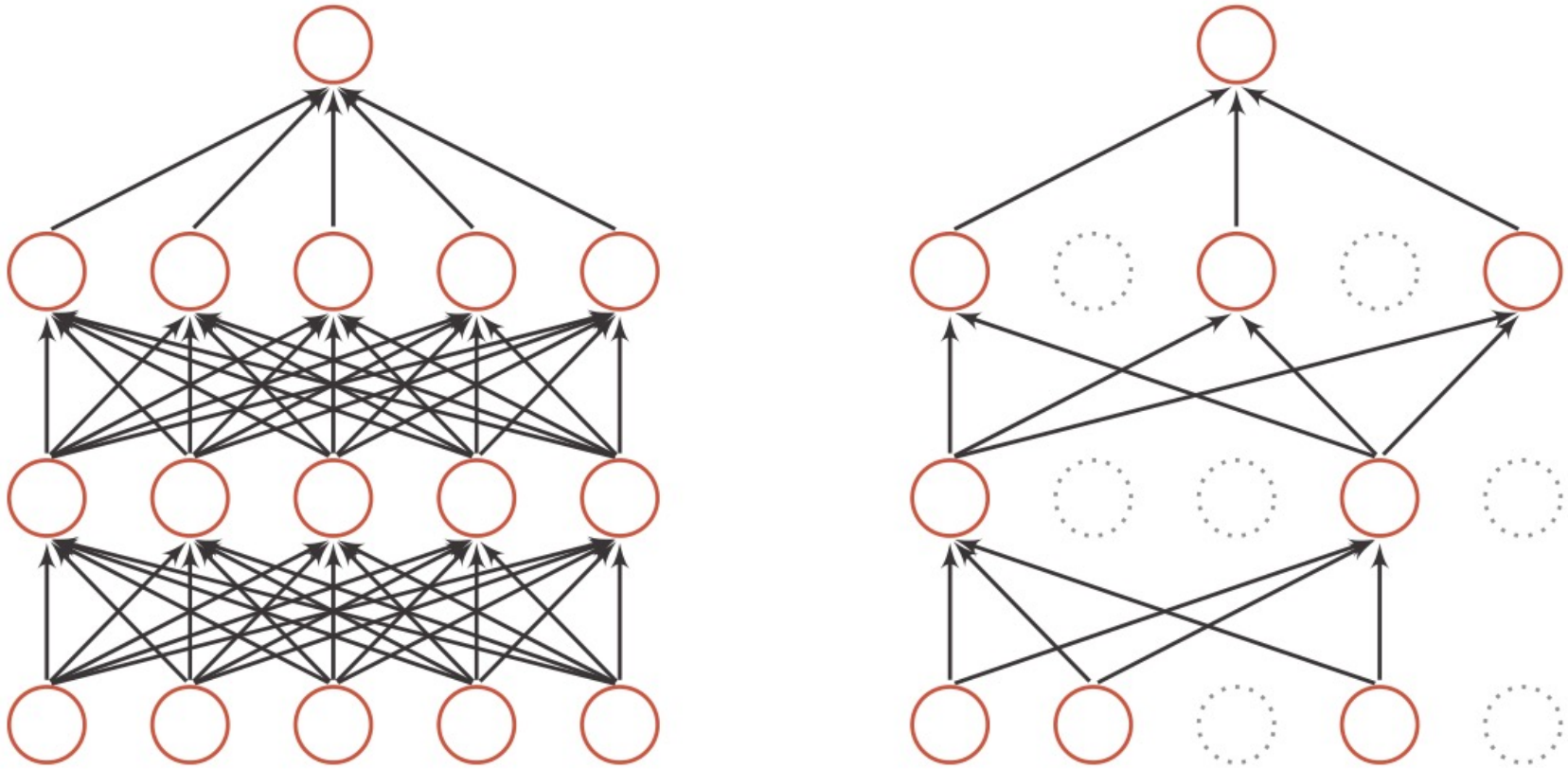
- 在标准的随机梯度下降中，权重衰减正则化和ℓ_2正则化的效果相同。
- 在较为复杂的优化方法（比如Adam）中，权重衰减和ℓ_2正则化并不等价。

提前停止

- 使用一个验证集来测试每一次迭代的参数在验证集上是否最优。如果在验证集上的错误率不再下降，就停止迭代。



丢弃法（Dropout Method）



(a) 标准网络　　(b) Dropout 后的网络

Dropout意义

- 集成学习的解释
  - 每做一次丢弃，相当于从原始的网络中采样得到一个子网络。如果一个神经网络有n个神经元，那么总共可以采样出2^n个子网络。
- 贝叶斯学习的解释

$$\mathbb{E}_{q(\theta)}[y] = \int_q f(\mathbf{x}, \theta) q(\theta) d\theta$$
$$\approx \frac{1}{M} \sum_{m=1}^{M} f(\mathbf{x}, \theta_m),$$

  - 其中$f(\boldsymbol{x}, \theta_m)$为第m次应用丢弃方法后的网络。

数据增强

- 图像数据的增强主要是通过算法对图像进行转变，引入噪声等方法来增加数据的多样性。
- 图像数据的增强方法：
  - 旋转（Rotation）：将图像按顺时针或逆时针方向随机旋转一定角度；
  - 翻转（Flip）：将图像沿水平或垂直方法随机翻转一定角度；
  - 缩放（Zoom In/Out）：将图像放大或缩小一定比例；
  - 平移（Shift）：将图像沿水平或垂直方法平移一定步长；
  - 加噪声（Noise）：加入随机噪声。

标签平滑

- 在输出标签中添加噪声来避免模型过拟合。
- 一个样本x的标签一般用onehot向量表示

$$\mathbf{y} = [0, \cdots, 0, 1, 0, \cdots, 0]^{\mathrm{T}}$$

硬目标（Hard Targets）

- 引入一个噪声对标签进行平滑，即假设样本以ϵ的概率为其它类。平滑后的标签为

$$\tilde{\mathbf{y}} = [\frac{\epsilon}{K-1}, \cdots, \frac{\epsilon}{K-1}, 1-\epsilon, \frac{\epsilon}{K-1}, \cdots, \frac{\epsilon}{K-1}]^{\mathrm{T}}$$

软目标（Soft Targets）

编程实现

- 代码学习（/course357文件夹中）：
  - 10.1-网络优化与正则化
  - 10.2-GoogLeNet(Flower)-pretrain

- 代码练习
  - 10.3-Flower-pretrain